WRAPER FAZA 1 SEKCJA 1 i 2

KONSTRUKCJA WRAPERA SEKCJI LICZB I SEKCJI 3D

KINETIC — Integracja z HERO/STATS przez BRIDGE (Snap Gate + const l)

Wersja: 1.1 (do scalenia z Konstytucją)
Cel: HERO/STATS "ucieka", a KINETIC spod spodu zaczyna żyć zanim HERO/STATS zniknie. Ma być crossover (bez pustki) i zero naruszenia snapów/timingów Kinetic.

1) Zasada naczelna

Jeden pinned ScrollTrigger + jeden timeline (`pinnedTl`) steruje *dwoma warstwami* w tym samym wrapperze:

- `stats-layer` (HERO/STATS) na wierzchu (z-index wyżej)
- `kinetic-layer` pod spodem (z-index niżej), startowo "niewidoczny" i wchodzący w BRIDGE

To jest reżyseria "After Effects": pin zamraża wrapper, a my animujemy wewnątrz.

NIE robimy dwóch osobnych pinów / dwóch ScrollTriggerów do styku (ryzyko jittera i "fake bottom").

---

2) Stałe i definicje (nie ruszać)

KINETIC jest kanoniczne w "unitach". Całe Kinetic ma stałą strukturę i zależności.

```
const KINETIC_U      = 23;      // święte

const SCROLL_KINETIC = 3526;    // px

const PX_PER_U       = 153.3;   // px/unit (SCROLL_KINETIC / KINETIC_U)



// BRIDGE length in timeline units (elastyczny wstęp)

const svh = /* small viewport height (stabilne) */;
```

const I = KINETIC_U * (svh / SCROLL_KINETIC); // const I = BRIDGE_U

window.BRIDGE_I = I;

Interpretacja: BRIDGE ma długość `I` jednostek timeline i skaluje się z wysokością viewportu.
KINETIC zaczyna się dopiero od `I`. Wszystko po `I` jest "strefą kinetyczną".

Snap points w unitach (kanon)

const SNAP_UNITS = [0, 3.5, 9.5, 23.0].map(u => u + I);

// 0+I → SNAP0

// 3.5+I → SNAP1

// 9.5+I → SNAP2

// 23+I → SNAP3 (Gravity Drop)

Zakaz: nie zmieniaj `KINETIC_U`, `SCROLL_KINETIC`, `blob1 duration = 23.69` i struktur zależnych. To jest "nuclear option".

---

3) Mapa timeline (gdzie wolno dotykać)

Twoja strefa integracji to tylko BRIDGE: `0 → I`.
Po `I` NIE dotykasz niczego poza podpięciem snap config / helperów.

0.00 BRIDGE START (Twoja strefa)

- HERO/STATS: ucieka (transformy, alpha)

- KINETIC: rodzi się pod spodem (overlay/bloby/Block1)

- Particle "!" formuje się (osobny canvas)

I BRIDGE END = start KINETIC zone

I+0.0 SNAP0 (ColorWave + blob keyframes)

I+3.5 SNAP1 (Block 1→2)

I+9.5 SNAP2 (Block 2→3 + eksplozja)

I+10.80 💥 Eksplozja + NIGDY glow

I+13.0 Cylinder intro

I+23.0 SNAP3 (Gravity Drop / finał)

---

4) Snap Gate (obowiązkowe) — BRIDGE bez snapa, KINETIC z agresywnym snapem

Problem: snap jest “jeden na cały ScrollTrigger”, więc bez bramki kompresuje BRIDGE.

Rozwiązanie: custom `snapTo()` w 3 strefach:

- **BRIDGE: brak snapa (`return p`) → scrub 1:1**
- **GRAB ZONE: końcówka bridge'a → scroll w dół "wciąga" do `BRIDGE_END`, scroll w górę nie snapuje**
- **KINETIC: normalny directional snapping po punktach snapów**

Implementacja (kanon):

```
// 1) Po zbudowaniu pinnedTl musisz znać total duration (w unitach timeline)

const totalU = pinnedTl.duration(); // ~ l + 23.xx



const BRIDGE_END = l / totalU;        // progress 0..1 gdzie kończy się bridge

const GRAB_START = BRIDGE_END * 0.82;  // ostatnie ~18% bridge

const HYS       = Math.min(0.03, BRIDGE_END * 0.25);



// snap points (progress 0..1) liczone z unitów (albo z labeli – preferowane)

const snapProgress = SNAP_UNITS.map(u => u / totalU);

const snapDir = ScrollTrigger.snapDirectional(snapProgress);
```

```
scrollTrigger.snap = {

 inertia: false,        // wymusza deterministykę (szczególnie z Lenis)

 directional: true,

 delay: 0.10,

 duration: { min: 0.20, max: 0.60 },

 ease: "power2.out",

 snapTo: (value, self) => {

   const p = self.progress;

   const dir = self.direction || 1;



   // STREFA 1: BRIDGE (zero snapa)

   if (p < GRAB_START) return p;



   // STREFA 2: GRAB ZONE (domyka bridge tylko przy scroll w dół)
```

```
  if (p < BRIDGE_END + HYS) return (dir > 0) ? BRIDGE_END : p;



  // STREFA 3: KINETIC (pełne snapy)

  return snapDir(value, dir);

 }

};
```

Dlaczego `return p` działa: GSAP ma guard `endScroll !== scroll`. Gdy zwrócisz `p`, endScroll ≈ scroll i snap tween nie startuje.
Zakaz: nie używać `return undefined` (NaN).

---

5) Jak podpinasz HERO/STATS w BRIDGE (w praktyce)

W BRIDGE animujesz tylko rzeczy z HERO/STATS i ewentualnie "wejście" Kinetic pod spodem.

Zasady wydajności i stabilności (hard rules):

- nie animuj layoutu (`top/left/height`) → tylko `transform` (`yPercent/y`) + `autoAlpha/opacity`

- nie animuj `background-color` fullscreen → użyj overlay i animuj `opacity`

- **wrapper i jego rodzice: zero `transform/filter/perspective` (miny z pinned/fixed)**
- **`overflow:hidden` tylko na wrapperze, nie na łańcuchu 3D tekstów (tam `overflow: visible` / preserve-3d chain)**

Przykład (BRIDGE: 0 → I):

```
pinnedTl.to(heroContent, {
 autoAlpha: 0,
 yPercent: -10,
 duration: I * 0.7,
 ease: "power2.in"
}, 0); // w bridge zone NIE dodajesz +I
```

Żelazna reguła offsetów:

- **w BRIDGE (`0 → I`): pozycje w timeline bez `+I`**
- **po BRIDGE (`>= I`): wszystko co “należy do kinetyki” ma pozycje zawsze z `+I` (albo wynika z kanonicznych unitów / labeli)**

---

6) Viewport units i endPx (mobile-miny)

Wysokość sceny i wyliczenia muszą być stabilne na mobile:

- **CSS stage: `height: 100lvh` (fallback `100vh`)**
- **pozycje bloków: `svh`**
- **JS: l i endPx liczone z `svh`**

ABSOLUTNA ZASADA:
`endPx = svh + SCROLL_KINETIC` (bez dopinania "toolbar delta" — to się znosi, a dopinanie psuje osiągalność snap3).

---

7) Lenis / scroll-proxy (utwardzenie deterministyki)

- **ScrollTrigger jest "pasażerem", Lenis jest "kierowcą"**
- **`ScrollTrigger.update` na `lenis.on("scroll")`**
- **nie używać `normalizeScroll(true)` (kolizje)**
- **ważne utwardzenie: jeśli scroller/proxy daje float scroll (np. 1200.4), Snap Gate może odpalić snapa mimo `return p`. W adapterze/proxy zaokrąglaj scroll do int (`Math.round`) przy get/set.**

---

8) Init order i API (zostaje, ale doprecyzowane)

Kolejność:

1. `window.load` **(fonty)**
2. `initKinetic(container)` **→ buduje** `pinnedTl`**, podpina ScrollTrigger+SnapGate, eksportuje:**
   - `window.pinnedTl`
   - `window.BRIDGE_I`
3. **IIFE'y (Particle/Cylinder/Tunnel/Explosion) czekają na** `window.pinnedTl` **jak dotąd**

SPA cleanup:

window.kineticSection.kill();

window.kineticSection.pause();

window.kineticSection.resume();

kill() przed unmount zawsze.

---

9) Antywzorce (najkrótsza lista, ale zabija 90% regresów)

- ❌ **zmiana** `KINETIC_U / blob1 duration / SCROLL_KINETIC`

- ❌ dokładanie tweenów w kinetic bez `+I` (lub bez kanonicznych unitów/labeli)
- ❌ animacja layoutu (top/height) zamiast transform
- ❌ pełnoekranowy background-color tween zamiast overlay opacity
- ❌ transform na wrapperze/rodzicach pinned
- ❌ dwa niezależne pin'y na styku sekcji
- ❌ `return;` / `return undefined` w snapTo

---

10) Checklist przed merge (minimalny, ale twardy)

- `const I` istnieje i jest liczone z `svh` (bez własnych mnożników "bo fajniej")
- `KINETIC_U === 23`, `SCROLL_KINETIC === 3526`, `blob1 duration === 23.69`
- `pinnedTl.duration() ≈ I + 23.xx`
- SnapGate: BRIDGE nie snapuje (wolny scroll 1:1), kinetic snapuje agresywnie
- `endPx = svh + SCROLL_KINETIC` i snap3 jest osiągalny na mobile

- **wrapper i rodzice: brak transform/filter/perspective**
- **wszystkie nowe animacje w bridge: transform/alpha, zero layoutu**

Jeśli chcesz, mogę Ci teraz zrobić super-krótką wersję “1 ekran” (taki *TL;DR dla deva do powieszenia nad monitorem*) — ale powyższe to już jest “konstytucyjny” zapis, który minimalizuje ryzyko, że ktoś zepsuje Kinetic niechcący.

SEKCJA KINETIC 3D ZE ZNAKIEM ZAPYTANIA

— Briefing integracyjny

Dla dewelopera łączącego sekcję HERO/STATS z sekcją KINETIC

Wersja: 1.0 | Źródło: Konstytucja v4.2 + 34 sesje

CO ROBISZ

Sekcja poprzedzająca (HERO/STATS) ma uciekać z ekranu. Gdy jest w połowie wyjścia — sekcja KINETIC (pod spodem, przezroczysta) zaczyna swoją animację. Dwa różne czasy scrollu muszą się zsynchronizować BEZ zmiany istniejących snap pointów i timingów kinetyki.

SYSTEM BRIDGE — `const I`

To jest JEDYNY mechanizm łączenia. Nie wymyślaj nowego.

const I = KINETIC_U * (vh / SCROLL_KINETIC);

`I` to **zmienna ilość jednostek timeline** na początku — "elastyczny wstęp" PRZED kinetyczną animacją. Rośnie z viewport height.

vh=400 → I ≈ 2.61 jednostek
vh=700 → I ≈ 4.57 jednostek
vh=1000 → I ≈ 6.53 jednostek

Co to oznacza dla Ciebie: masz `I` jednostek timeline na fade-out sekcji HERO i fade-in kinetyki. Reszta animacji (`3.5+I`, `9.5+I`, `23.0+I`) NIE PRZESUWA SIĘ — one są zakotwiczone do `+I`.

Czego NIE wolno

Akcja	Dlaczego

Zmieniać `I` (mnożyć, zaokrąglać, przesuwać)	Desync blobów, snap pointów, eksplozji — WSZYSTKIEGO
Dodawać tweeny bez `+I` w kinetic zone	Pozycja absolutna z `+I` to **żelazna zasada**
Zmieniać `KINETIC_U` (23)	18 zależności do przeliczenia — NUCLEAR OPTION
Zmieniać duration blob1 (23.69)	Definiuje `pinnedTl.duration()` — zmiana niszczy mapping

---

TIMELINE — MAPA

```
UNIT      CO SIĘ DZIEJE                    SNAP
──────────────────────────────────────────────────────────
0.00      ← BRIDGE START (tu jest Twoja strefa)
          Hero fade-out, particle "!" formuje się
          Blob opacity fade-in (I-2)
I         ← BRIDGE END → Kinetic zone start
0.0+I     SNAP 0 — ColorWave, blob keyframes      [0]
3.5+I     SNAP 1 — Block 1→2 transition           [1]
9.5+I     SNAP 2 — Block 2→3, eksplozja           [2]
10.80+I   💥 Eksplozja + NIGDY glow
13.0+I    Cylinder intro
23.0+I    SNAP 3 — Gravity Drop, finał            [3]
──────────────────────────────────────────────────────────
```

Twoja strefa = 0 → I. Nie dotykasz niczego po `I`.

CO MUSISZ WIEDZIEĆ O BRIDGE ZONE (0 → I)

1. **Particle "!" formuje się podczas bridge.** Bridge mapping w Particle IIFE mapuje `0→I` na `0→FORM_END`. Nie koliduje z Twoim fade-outem — particle jest na osobnym canvasie (`z-index:3`).

2. **Blob opacity startuje przy `I-2`.** Bloby wchodzą 2 jednostki PRZED końcem bridge. Jeśli `I < 2` (mały viewport), bloby startują od razu.

3. **SNAP nie działa w bridge zone.** `snapTo` ma guard: `if (p < GRAB_START) return p` — w bridge zone scroll jest swobodny (1:1 scrub).

JAK PODŁĄCZYĆ HERO FADE-OUT

Opcja A: W tym samym pinnedTl (rekomendowana)

Hero elementy animujesz na pozycjach `0 → I`. Przykład:

```
// Hero content fade-out — zaczyna od razu, kończy przed bridge end
pinnedTl.to(heroContent, {
  autoAlpha: 0,
  y: -80,
  duration: I * 0.7,   // 70% bridge time
  ease: "power2.in"
}, 0);  // ← pozycja 0, BEZ +I (jesteśmy w bridge zone)
```

ZASADA: W bridge zone (0 → I) pozycje są BEZ `+I`. Po bridge — ZAWSZE z `+I`.

Opcja B: Osobny ScrollTrigger na sekcję HERO

Jeśli HERO ma własny scroll — koordynacja przez progress. Kinetic pin startuje gdy HERO jest w ~50% wyjścia. To jest bardziej złożone i wymaga synchronizacji dwóch ScrollTriggerów.

ZDERZENIE NAPISÓW — KONKRETY

Problem: napis jedzący od dołu może się zderzyć z "nie stanie się jej klientami" (block-3).

Pozycja "nie stanie się jej klientami"

DESKTOP (≥600px): Block-3 jest na `top:50%` + GSAP `yPercent:-50` = wycentrowany.

Viewport	.line font-size	line-height (×1.2)	Offset od góry bloku
1440px	40.3px	48.4px	~150px

1200px	34.6px	41.5px	~136px
900px	25.9px	31.1px	~110px
600px	20.8px	25.0px	~93px

Desktop wzór: `offset ≈ (header_font × 1.25 + 24px) + (line_height × 2)`

MOBILE (<600px): Block-3 na `top: calc(145px + 21.1svh)`.

Viewport	.line = 6.6vw	line-height	Offset (4 linie przed bold)
390px	25.7px	30.9px	~172px
375px	24.8px	29.7px	~167px
320px	21.1px	25.3px	~149px

Mobile wzór: `offset ≈ 48px + (line_height × 4)`

Wygaszaj nadjeżdżający napis 1 `line-height` PRZED progiem zderzenia — to daje ~25-48px marginesu (zależnie od viewport).

STAŁE — NIE RUSZAJ

KINETIC_U = 23 // jednostek kinetycznych
SCROLL_KINETIC = 3526 // px (23 × 153.3)
PX_PER_U = 153.3 // pikseli na jednostkę
blob1 duration = 23.69 // ŚWIĘTA KROWA — definiuje timeline
SNAP_UNITS = [0+I, 3.5+I, 9.5+I, 23.0+I]

VIEWPORT UNITS — PUŁAPKI MOBILE

CSS stage: height: 100lvh (fallback: 100vh)
CSS block positions: svh (stabilne z paskiem przeglądarki)
JS bridge I: svh-based
JS endPx: svh + SCROLL_KINETIC

Jednostka	Zachowanie	Użycie
`vh`	Zmienia się z toolbarem — **niestabilne**	Tylko jako fallback
`svh`	Small viewport (z paskiem) — **stałe**	Pozycje, bridge I, endPx
`lvh`	Large viewport (bez paska) — **stałe**	Stage height (gwarantuje scroll reach)

ABSOLUTNA ZASADA: `endPx = svh + SCROLL_KINETIC`. Nie dodawaj toolbar delty — znosi się w równaniu.

INICJALIZACJA — KOLEJNOŚĆ

1. window.load (fonty gotowe)
2. init(container) — buduje pinnedTl, eksportuje do window.pinnedTl
3. Particle IIFE — czeka na window.pinnedTl (setInterval)
4. Cylinder IIFE — czeka na window.pinnedTl
5. Tunnel IIFE — czeka na window.particleQmark.state
6. Explosion — trigger w timeline na 10.80+I

Jeśli Twoja sekcja HERO potrzebuje `pinnedTl` — czekaj na `window.pinnedTl`.

WINDOW API — CO MASZ DO DYSPOZYCJI

```
window.pinnedTl                    // Główny timeline
window.pinnedTl.progress()         // 0→1 aktualny postęp
window.pinnedTl.scrollTrigger      // ScrollTrigger instance
window.BRIDGE_I                    // wartość I (eksportowana)
window.particleQmark.state.formProgress  // 0→1 formacja "!"
```

CLEANUP — OBOWIĄZKOWY

Kinetic ma system `kill()` / `pause()` / `resume()`:

```
window.kineticSection.kill();    // Pełny cleanup
window.kineticSection.pause();   // Zamroź (ticker + timeline)
window.kineticSection.resume();  // Odblokuj
```

W SPA/routing: `kill()` PRZED odmontowaniem. Kinetic sam wywołuje `kill()` przed re-init.

ANTYWZORCE — SZYBKA LISTA

❌ Nie rób	✅ Rób tak
`pinnedTl.to(el, {}, "+=2")`	`pinnedTl.to(el, {}, 5.5 + I)`
`pinnedTl.from(el, {...})`	`pinnedTl.fromTo(el, {...}, {...})`
`window.innerWidth < 600`	`getComputedStyle(el).display !== 'none'`
rAF + timeline na TYM SAMYM elemencie	Timeline steruje parametrami, rAF renderuje piksele
`overflow: hidden` na ancestors tekstu	`overflow: visible` (preserve-3d chain)
Duration tweena > 23.69	Zawsze < blob1 duration

CHECKLIST PRZED MERGE

□ grep "const I" — musi istnieć, niezmieniony
□ grep "KINETIC_U" — musi być 23
□ pinnedTl.duration() w konsoli — ~29-30 (I+23)
□ Nowe tweeny w bridge zone (0→I): BEZ +I
□ Nowe tweeny w kinetic zone: ZAWSZE z +I
□ Żaden nowy tween nie jest dłuższy niż 23.69
□ gsap.set pre-pozycjonowanie = stan początkowy
□ Test: wolny scroll (nie snap!) — płynne przejście?
□ Test: mobile z toolbarem — snap3 osiągalny?
□ Test: desktop resize — brak crash?
□ z-index nowych elementów sprawdzony w mapie warstw

WRAPER FAZA 2 SEKCJA 2 i 3

Konstytucja: przejście Kinetic → WYCHODZĄ (wersja dla deva)

1) Architektura DOM (kanon)

Wciąż jeden wrapper i warstwy (jak bridge z liczb).

```
<div id="handoff-wrapper">

 <div id="kinetic-layer"></div>     <!-- pinned, scrub + snap -->

 <div id="outro-layer"></div>       <!-- WYCHODZĄ, startuje poza kadrem -->

</div>
```

CSS (najważniejsze zasady):

- `#handoff-wrapper` **i jego rodzice: zero** `transform/filter/perspective/contain` **(miny dla pinu i fixed)**
- **warstwy:** `position:absolute; inset:0;`
- `outro-layer` **start:** `transform: translateY(100%);` **(albo** `yPercent: 100`**)**
- **tło / overlay: identyczne na końcu Kinetic i początku WYCHODZĄ (żeby nie było “flash’a”)**

Jeśli “WYCHODZĄ” ma elementy `position: fixed` (canvas/overlay itp.) i muszą być *prawdziwie viewport-fixed*, to nie mogą być zagnieżdżone wewnątrz pinned wrappera. Wtedy robi się portal na `body` (global fixed layer) i steruje tylko opacity/z-index. Jeśli “WYCHODZĄ” jest już przygotowana pod wrapper (jak piszesz) — super, jedziemy w środku.

---

2) Timeline: co jest nienaruszalne

Wszystko do eksplozji + “98% ludzi” zostaje *pixel-invariant*.
Nie ruszamy snapów i timingów części kinetycznej.

Klucz: zamiast “Gravity Drop”, repurposujemy ogon timeline’u na:

- **freeze (brak zmian w kinetic-layer)**
- **wjazd outro-layer od dołu**
- **ewentualny micro-fade/overlay żeby przejęcie wyglądało czysto**

Masz już stałe momenty:

- `SNAP0 = 0 + I`
- `SNAP1 = 3.5 + I`
- `SNAP2 = 9.5 + I`
- `EXPLODE = 10.80 + I` ***(tu kończy się “akcja”)***
- `SNAP3 = 23.0 + I` ***(teraz to będzie “handoff complete”)***

Nowy kontrakt:

- **od `EXPLODE` do `SNAP3` Kinetic jest *zamrożony w kadrze***

- **w tym samym oknie `outro-layer` wjeżdża od dołu i przejmuje ekran**
- **pin kończysz zaraz po `SNAP3` (bez długiego “martwego ogona”)**

---

3) Implementacja: ogon timeline’u (minimalny, kuloodporny)

Najprościej:

```
// OUTRO wjeżdża w oknie po eksplozji

const OUTRO_IN_START = 10.80 + l;

const OUTRO_IN_DUR   = (23.0 - 10.80); // wykorzystujemy istniejące units do SNAP3



// 1) Zamrożenie Kinetic = po prostu NIC dalej nie animujesz na kinetic-layer

// (wszystko po EXPLODE to już outro + ewentualnie overlay)



// 2) Outro wjeżdża od dołu (transform-only)

pinnedTl.fromTo(outroLayer,

 { yPercent: 100, autoAlpha: 1 },   // autoAlpha opcjonalnie jeśli masz przezroczystość

 { yPercent: 0, duration: OUTRO_IN_DUR, ease: "none" },
```

OUTRO_IN_START

);

// 3) Jeśli trzeba "maskować" mikroróżnice tła: overlay opacity

pinnedTl.to(outroOverlay, { autoAlpha: 1, duration: OUTRO_IN_DUR, ease: "none" }, OUTRO_IN_START);

To jest odwracalne (scrub), nie ma eventów “na granicy”, więc user może mielić góra/dół do woli.

---

4) Snap: musisz dodać drugą bramkę (żeby snap nie przeszkadzał w wjeździe outro)

Jeżeli zostawisz agresywny snap bez zmian, to po SNAP2 może Ci “ciągnąć” z powrotem do 9.5+l i utrudniać płynne wjechanie outro.

Rozwiązanie: po EXPLODE wyłącz snap (return p) aż do końcowej “strefy domknięcia”, która łapie do SNAP3.

Schemat stref w `snapTo`:

1. BRIDGE (jak masz) → `return p`
2. KINETIC (do EXPLODE) → normalne `snapDir`
3. OUTRO REVEAL (EXPLODE → prawie SNAP3) → `return p` (żeby wjazd był 1:1)
4. END GRAB (końcówka przed SNAP3) → scroll w dół dociąga do SNAP3

Pseudo:

```
const totalU = pinnedTl.duration();



const P_EXPLODE = (10.80 + l) / totalU;

const P_END     = (23.0  + l) / totalU;



const END_GRAB_START = P_END - 0.06; // ostatnie ~6% progressu jako
"domknięcie"



snapTo: (value, self) => {

 const p   = self.progress;

 const dir = self.direction || 1;



 // (A) BRIDGE gate zostaje jak wcześniej (GRAB_START/BRIDGE_END/HYS)



 // (B) KINETIC snapping tylko do eksplozji

 if (p >= BRIDGE_END + HYS && p < P_EXPLODE) return snapDir(value, dir);
```

```
// (C) OUTRO reveal: zero snapa

if (p < END_GRAB_START) return p;



// (D) domknięcie do SNAP3 przy scroll w dół

return (dir > 0) ? P_END : p;

}
```

To jest dokładnie ta sama filozofia co Snap Gate w bridge: tam, gdzie ma być "filmowe przejście", nie ma magnetyzmu.

---

5) Koniec pinu (najważniejszy detal wizualny)

Pin ma się kończyć tuż po tym, jak outro dojedzie do stanu "100% przejęcia". Nie zostawiaj długiego ogona, bo ogon uwielbia ujawniać "jolt".

Praktyczna zasada:

- `ScrollTrigger.end` **= taka długość, żeby** `SNAP3` **był *prawie przy końcu*, a po nim 0–2% bufora.**

---

6) Kasacja artefaktów synchronizacji (to nadal obowiązuje)

Te rzeczy muszą zniknąć:

- `<div style="height:2100px">` **spacer**
- `margin-top:-220vh` **itd.**

Sekcja WYCHODZĄ ma iść bezpośrednio po pin-spacerze ScrollTriggera (albo być warstwą w wrapperze, jak teraz).

---

7) Jedyny realny “demon” przy tym podejściu

Jeśli sekcja WYCHODZĄ ma logikę opartą o surowy `window.scrollY` (velocity detection, progi), to podczas pinu `scrollY` rośnie i może ją “odpalić” zanim będzie widoczna.

Kuloodporne zabezpieczenie:

- **aktywuj jej logikę dopiero, gdy sekcja realnie jest w viewport (observer ST / IO):**

```
let isActive = false;

ScrollTrigger.create({

 trigger: "#wychodza-section",

 start: "top bottom",

 onEnter: () => isActive = true,

 onLeaveBack: () => isActive = false,

});
```

TL;DR dla deva (1 ekran)

- **Nie robimy żadnego nowego pinu do handoffu.**
- **Kinetic do EXPLODE bez zmian.**
- **Po EXPLODE Kinetic zamraża kadr, a WYCHODZĄ wjeżdża od dołu (transform-only) aż do `SNAP3`.**
- **Snap: po EXPLODE snap off (return p), a końcówka dociąga do SNAP3.**
- **Pin kończy się tuż po SNAP3.**
- **Żadnych spacerów/margin-hacków.**

Jeśli wkleisz mi minimalny fragment aktualnego `pinnedTl` (same label/units + gdzie teraz stoi “98% ludzi”), dopasuję Ci konkretnie: który unit traktujemy jako EXPLODE_FREEZE i jakie dokładnie liczby dać do bramek (P_EXPLODE, END_GRAB_START), tak żeby to było 1:1 z tym co macie w kodzie.

SEKCJA WYCHODZĄ KONSTYTUCJA PRZYGOTOWANIA

Wave-Wrap — Instrukcja Integracji (Portal Body)

Dlaczego poza sekcją?

`position: fixed` wewnątrz elementu z `transform` (np. GSAP pin, smooth scroll wrapper) przestaje być relative do viewportu — staje się relative do rodzica. Wave-wrap wyjeżdża z ekranu razem z sekcją.

Przeniesienie na body level eliminuje ten problem. Żaden `transform` z GSAP na żadnym wrapperze nie złamie `position: fixed`.

Struktura HTML — gdzie umieścić wave-wrap

html

```
<body>

  <!-- ... sekcje wcześniejsze ... -->

  <!-- BRIDGE WRAPPER: pinowana sekcja kinetic -->
  <div id="bridge-wrapper">
    <!-- kinetic section content, GSAP pin -->
  </div>
```

```
<!-- ═══════════════════════════════════════════════════════════════════════ -->

<!-- WAVE-WRAP: PORTAL BODY — bezpośredni child <body>  -->

<!-- MUSI BYĆ PRZED sekcją blok-4-5 w DOM              -->

<!-- ═══════════════════════════════════════════════════════════════════════ -->

<div id="blok-4-5-wave-wrap">

  <svg id="blok-4-5-wave-svg" viewBox="0 0 100 100" preserveAspectRatio="none">

    <path class="wave-path"></path>

    <path class="wave-path"></path>

    <path class="wave-path"></path>

    <path class="wave-path"></path>

  </svg>

</div>



<!-- SEKCJA: blok-4-5 -->

<div id="blok-4-5-section">

  <!-- ... cała zawartość sekcji ... -->

</div>
```

```
  <!-- ... sekcje następne ... -->



</body>
```

Zasady umieszczenia

1. Bezpośredni child `<body>` (lub głównego wrappera strony, jeśli nie ma na nim `transform`)
2. PRZED `#blok-4-5-section` w kolejności DOM (naturalny z-index stacking)
3. NIE wewnątrz żadnego wrappera z `transform`, `will-change`, `filter`, `contain`, `perspective`, `backdrop-filter`
4. NIE wewnątrz `#blok-4-5-section`
5. NIE wewnątrz `#bridge-wrapper` ani żadnego pinowanego elementu

CSS wave-wrap

css

```
/* Selektor po ID — nie zależy od parenta */

#blok-4-5-wave-wrap {

   position: fixed;

   inset: 0;

   z-index: 0;         /* nad kinetic (z-index:-1), pod sekcją (z-index:2) */

   pointer-events: none;

   overflow: hidden;

   visibility: hidden;   /* startowo ukryty — autoAlpha pattern */
```

```
    opacity: 0;
    transition: none !important; /* GSAP kontroluje autoAlpha — blokuj CSS transitions */
}

#blok-4-5-wave-svg {
    width: 100%;
    height: 100%;
    pointer-events: none;
    position: absolute;
    top: 0;
    left: 0;
    z-index: 1;
    transform: translateZ(0); /* GPU layer isolation — Safari fix */
}

#blok-4-5-wave-svg .wave-path:nth-of-type(1) { fill: #f5ede1; }
#blok-4-5-wave-svg .wave-path:nth-of-type(2) { fill: #ebe3d1; }
#blok-4-5-wave-svg .wave-path:nth-of-type(3) { fill: #edd9d0; }
```

```
#blok-4-5-wave-svg .wave-path:nth-of-type(4) { fill: #f7f6f4; }
```

KRYTYCZNE reguły CSS z researchu:

1. `transition: none !important` — jeśli globalny CSS ma `transition: all` lub `transition: opacity`, to GSAP autoAlpha wchodzi w konflikt z CSS transition. Blokujemy bezwarunkowo.
2. `transform: translateZ(0)` na SVG — wymusza osobną warstwę GPU. W Safari skomplikowane SVG paths obok `position: fixed` mogą powodować glitche GPU bez tego.
3. Brak `contain` — `contain: paint/layout` na wave-wrap złamałoby `position: fixed` children (gdyby były). Nie dodawaj contain.

Z-index stacking (od dołu do góry) — ZWERYFIKOWANE

Warstwa	Element	z-index	position
0 (dno)	Bridge/kinetic (zamrożona klatka)	-1	fixed (pin)
1	wave-wrap	0	fixed
2	#blok-4-5-section	2	relative
3	Treść sekcji (section)	1 (w kontekście parenta)	relative

KRYTYCZNE z researchu:

- `position: fixed` ZAWSZE tworzy stacking context (od Chrome 22, Safari 18+, Firefox)
- `position: relative` z jawnym `z-index` tworzy stacking context
- Różne z-index = deterministyczne — DOM order irrelewantny
- Bridge-wrapper/kinetic MUSI mieć z-index niższy niż 0 (np. -1)
- GSAP ScrollTrigger pin NIE nadaje automatycznie z-index — musisz sam kontrolować
- Pin-spacer może nie kopiować z-index z pinowanego elementu — jawnie nadaj z-index

---

Widoczność — autoAlpha, NIGDY display:none

Dlaczego nie display:none?

`display:none` usuwa element z render tree. SVG paths nie są renderowane. Animacja biegnie w JS ale jest niewidoczna. Przy ponownym `display:''` widzisz skok do stanu końcowego.

Jak działa autoAlpha?

`gsap.set(el, { autoAlpha: 0 })` ustawia `opacity: 0` + `visibility: hidden`.
`gsap.set(el, { autoAlpha: 1 })` ustawia `opacity: 1` + `visibility: visible`.

Element ZAWSZE jest w render tree. Przeglądarka zawsze renderuje SVG paths. Animacja jest widoczna.

Kiedy co się odpala?

Moment	Akcja
Strona się ładuje	CSS: `visibility: hidden; opacity: 0` — ukryty
`playOpen()` startuje	`gsap.set(waveWrap, { autoAlpha: 1 })` — widoczny
`playOpen()` kończy	paths zakrywają viewport, `isOpened = true`
`playClose()` startuje	Nic — wave-wrap pozostaje `autoAlpha: 1`, animacja gra
`playClose()` kończy (`currentTime ≤ 0`)	`gsap.set(waveWrap, { autoAlpha: 0 })` — ukryty

ScrollTriggery — 2 zamiast 3

NIE MA visibility gate. Żadnego trzeciego ScrollTriggera.

stWaveOpen:

trigger: '#blok-4-5-ludzie-wchodza'

start: 'top 85%'

onEnter → playOpen()

stWaveClose:

trigger: '#blok-4-5-voidSectionWrapper'

start: 'bottom bottom'

onLeaveBack → playClose()

Dlaczego nie visibility gate?

Visibility gate (`display:none` przy wyjściu sekcji z viewportu) zabijał animację `playClose` w trakcie trwania. Sekcja wyjeżdżała z ekranu ~50ms po starcie `playClose`, a `display:none` ubijał renderowanie. Użytkownik widział skok.

Bez visibility gate: wave-wrap z `autoAlpha: 0` nie blokuje niczego (`pointer-events: none`, `opacity: 0`, `visibility: hidden`). Nie potrzebuje `display:none`.

Safety-bg — stały kolor tła od ilustracji w dół

Realizacja przez CSS na poszczególnych elementach (bez dodatkowego wrapper diva):

css

```
/* "Możemy to zmienić" — ciepłe tło zawsze */

#blok-4-5-section .text-on-illustration-bottom {

    background-color: #f7f6f4;

}




/* Block-5: Konwersja, realne pieniądze, itd. */

#blok-4-5-section #blok-4-5-block-5-content {

    background-color: #f7f6f4;

}
```

Spacer między illustration-container a block-5-content: `background-color: #f7f6f4` **inline.**

Zasada: Każdy element od ilustracji w dół ma własne ciepłe tło. Nie potrzeba wrapper diva — unikamy problemów z nesting. Góra sekcji ("Ludzie wchodzą", "i wychodzą") zostaje przezroczysta — tam fala pracuje.

Pliki do integracji

Plik	Co zawiera	Gdzie

`blok-4-5-standardized.html`	Cała sekcja + wave engine + triggery	Standalone test shell
`WAVE-WRAP-INTEGRACJA.md`	Ten dokument	Spec integracyjna

Co usunąć z test shell przy integracji

1. `<div style="position:fixed; inset:0; background:#2a2130; z-index:0;">` — zastąpiony przez prawdziwy kinetic/bridge
2. `<div style="height: 100vh;">` — zastąpiony przez pin-spacer GSAP z bridge wrappera
3. CSS w `<style>` w `<head>` (fonty, reset) — już istnieje w globalnych stylach

Co przenieść

1. Wave-wrap HTML → bezpośrednio przed `#blok-4-5-section` w body
2. Wave-wrap CSS → do globalnego CSS (selektory po ID, nie zależą od parenta)
3. Wave engine JS (`initWave()`) → zostaje w `<script>` sekcji, `$id('blok-4-5-wave-wrap')` znajdzie element na body level

Wave Animation — Lokalizacja i Instrukcja dla Programisty

Gdzie jest animacja?

Cała animacja fali (kipiel engine) jest w pliku `blok-4-5-standardized.html`. Składa się z trzech elementów które trzeba wyciągnąć osobno:

---

1. HTML wave-wrap (linie ~52-60)

html

4. `<!-- WAVE REVEAL: body-level portal — outside section to avoid containing block issues -->`

```
5.  <div class="blok-4-5-wave-wrap" id="blok-4-5-wave-wrap">
6.    <svg class="blok-4-5-wave-overlay" id="blok-4-5-wave-svg" viewBox="0 0 100 100"
    preserveAspectRatio="none">
7.      <path class="wave-path"></path>
8.      <path class="wave-path"></path>
9.      <path class="wave-path"></path>
10.     <path class="wave-path"></path>
11.   </svg>

</div>
```

Gdzie to umieścić w produkcji:

```
12. <body>
13.   ...
14.   <!-- bridge-wrapper z pinowaną sekcją kinetic -->
15.   <div id="bridge-wrapper">...</div>
16.
17.   ═══▶ TUTAJ: wave-wrap ◀═══
18.
19.   <!-- sekcja blok-4-5 -->
20.   <div id="blok-4-5-section">...</div>
21.   ...

</body>
```

KRYTYCZNE ZASADY:

- **Wave-wrap MUSI być direct child `<body>` (lub wrappera BEZ transform/will-change/filter/contain)**
- **Wave-wrap MUSI być PRZED `#blok-4-5-section` w DOM order**
- **Wave-wrap NIE MOŻE być wewnątrz `#blok-4-5-section`**
- **Wave-wrap NIE MOŻE być wewnątrz `#bridge-wrapper`**
- **Wave-wrap NIE MOŻE być wewnątrz żadnego elementu z `transform` (nawet `transform: none`)**

Z-INDEX STACKING (zweryfikowane deep research 2026-02-24):

```
22. Bridge/kinetic:    z-index: -1  (position: fixed via GSAP pin)
23. Wave-wrap:         z-index:  0  (position: fixed)

#blok-4-5-section: z-index:  2  (position: relative)
```

- **GSAP pin NIE nadaje automatycznie z-index — musisz go jawnie ustawić na bridge-wrapper**
- **Pin-spacer może nie kopiować z-index — jawnie nadaj na pinowanym elemencie**
- **Wszystkie trzy z-index MUSZĄ być różne — gwarantuje deterministyczny stacking**

Dlaczego: `position: fixed` na wave-wrap musi być relative do viewportu. Jeśli jakikolwiek przodek ma `transform`, `will-change: transform`, `filter`, `contain: paint/layout` — to `position: fixed` łamie się i staje relative do tego przodka zamiast viewportu.

---

2. CSS wave-wrap (w `<style>` sekcji, linie ~660-680)

Szukaj komentarza: `/* --- WAVE REVEAL (scroll-driven bg transition) --- */`

```
css

#blok-4-5-wave-wrap {
    position: fixed;
    inset: 0;
    z-index: 0;
    pointer-events: none;
    overflow: hidden;
    visibility: hidden;   /* startowo ukryty */
    opacity: 0;           /* autoAlpha pattern */
    transition: none !important; /* GSAP controls autoAlpha — block CSS transitions */
}

#blok-4-5-wave-svg {
    width: 100%; height: 100%;
    pointer-events: none;
    position: absolute;
    top: 0; left: 0;
    z-index: 1;
    transform: translateZ(0); /* GPU layer isolation — Safari SVG fix */
}

#blok-4-5-wave-svg .wave-path:nth-of-type(1) { fill: #f5ede1; }
#blok-4-5-wave-svg .wave-path:nth-of-type(2) { fill: #ebe3d1; }
#blok-4-5-wave-svg .wave-path:nth-of-type(3) { fill: #edd9d0; }

#blok-4-5-wave-svg .wave-path:nth-of-type(4) { fill: #f7f6f4; }
```

Gdzie to umieścić:

W globalnym CSS lub w osobnym pliku. Selektory są po ID — nie zależą od parenta. UWAGA: Te selektory celowo NIE mają prefiksu `#blok-4-5-section` bo wave-wrap jest POZA sekcją. UWAGA: `transition: none !important` — jeśli globalny CSS ma `transition: all` lub `transition: opacity`, to bez tego GSAP autoAlpha wchodzi w konflikt z CSS transition i powoduje opóźnienia/zacięcia.

---

3. JS — silnik kipiel + ScrollTriggery (w `<script>` sekcji)

Szukaj komentarza: `// === WAVE REVEAL (Kipiel Engine) ===` Cały blok to IIFE: `(function initWave() { ... })();`

Granice kodu do wyciągnięcia:

START: `(function initWave() {` END: `})();` (zamknięcie IIFE, przed `var stUnderline = ScrollTrigger.create`)

Co jest w środku (w kolejności):

1. Pobranie elementów DOM:

js

```
47.  var waveWrap = $id('blok-4-5-wave-wrap');

     var wavePaths = waveWrap.querySelectorAll('.wave-path');
```

`$id()` to helper zdefiniowany wcześniej w tym samym `<script>`: `function $id(id) { return document.getElementById(id); }`

2. Kipiel engine — konfiguracja:
   - Stałe matematyczne (PI, HALF_PI)
   - Parametry krzywych (segments, control points, delay points)
   - 4 funkcje easing (elasticOut, backOut, power3Out) — po jednej na path
   - `PER_PATH[]` — delay + duration per path (staggered: 0ms, 90ms, 200ms, 340ms)
   - `maxTime` — ~1310ms (czas pełnej animacji)
3. Kipiel engine — render:
   - `updatePath(time, duration, pathIdx)` — generuje string SVG `d` attribute
   - `render(now)` — wywoływana co frame przez gsap.ticker
     - Liczy elapsed time
     - Aktualizuje currentTime (do przodu lub do tyłu)
     - Wywołuje updatePath dla każdego z 4 pathów
     - Na zakończeniu playClose (currentTime ≤ 0): `gsap.set(waveWrap, { autoAlpha: 0 })`
4. Ticker management:
   - `startTicker()` — dodaje render do gsap.ticker
   - `stopTicker()` — usuwa render z gsap.ticker
5. playOpen() / playClose():
   - `playOpen()`:
     - Guard: `if (isOpened && !isAnimating) return;`
     - `gsap.set(waveWrap, { autoAlpha: 1 })` — pokazuje wave-wrap

        - `reverse = false` → animacja od dołu do góry (zakrywa kinetic)
    - `playClose()`:
        - Guard: `if (!isOpened && !isAnimating) return;`
        - `reverse = true` → animacja od góry do dołu (odsłania kinetic)
        - Ukrycie (`autoAlpha: 0`) dzieje się w `render()` po zakończeniu
6. Inicjalizacja paths:
    - Generuje D_ZERO (ścieżka w pozycji "dół" = niewidoczna)
    - Ustawia D_ZERO na wszystkich 4 pathach
7. ScrollTriggery (2 sztuki):

js

```
// OPEN: "Ludzie wchodzą" wchodzi z dołu viewportu, mija 15% od dołu
ScrollTrigger.create({
  trigger: '#blok-4-5-ludzie-wchodza',
  start: 'top 85%',
  onEnter: function() { playOpen(); }
});

// CLOSE: user cofa, "i wychodzą" opuszcza dolną krawędź ekranu
ScrollTrigger.create({
  trigger: '#blok-4-5-voidSectionWrapper',
  start: 'bottom bottom',
  onLeaveBack: function() { playClose(); }

});
```

Zależności JS:

- GSAP 3.12+ (gsap, ScrollTrigger)
- `$id()` helper (lub zamień na `document.getElementById`)
- `gsapInstances[]` — tablica do której push'owane są ScrollTriggery (do kill() przy cleanup)

Jak to działa — flow scroll

User scrolluje W DÓŁ:
————————————————————
1. Widzi zamrożoną klatkę kinetic (ciemna, kolorowa)
2. Sekcja blok-4-5 wjeżdża od dołu (background: transparent)
3. Tekst "Ludzie wchodzą na stronę..." pojawia się
4. Gdy górna krawędź tekstu mija 85% viewportu od góry:
   → onEnter → playOpen()
   → gsap.set(autoAlpha: 1) — wave-wrap widoczny
   → SVG paths animują się od dołu do góry (~1.3 sekundy)

69. → 4 warstwy ciepłych kolorów zakrywają ciemną klatkę
70. → Ostatnia warstwa (#f7f6f4) = kolor tła strony
71. 5. Tekst jest teraz na ciepłym tle — czytelny
72. 6. User scrolluje dalej: ilustracja, "Możemy to zmienić", itd.
73.
74. User scrolluje W GÓRĘ (cofa się):
75. ──
76. 1. Treść sekcji cofa się w dół
77. 2. Gdy dolna krawędź "i wychodzą" opuszcza dolną krawędź viewportu:
78. → onLeaveBack → playClose()
79. → SVG paths animują się od góry do dołu (~1.3 sekundy)
80. → Ciepłe warstwy schodzą w dół, odsłaniając ciemną klatkę kinetic
81. → Po zakończeniu: gsap.set(autoAlpha: 0) — wave-wrap ukryty
82. 3. User wraca do sekcji kinetic
83.
84. Powtórny scroll W DÓŁ:
85. ──────────────────────────────
86. 1. "Ludzie wchodzą" znowu mija 85% → onEnter → playOpen()

2. Cały cykl powtarza się bez ograniczeń

Safety-bg — ochrona koloru tła

Od ilustracji w dół ZAWSZE jest ciepłe tło `#f7f6f4` jako zabezpieczenie. To jest realizowane przez CSS `background-color` na:

- `.text-on-illustration-bottom` — "Możemy to zmienić"
- Spacer div (inline style)
- `#blok-4-5-block-5-content` — "Konwersja!" i reszta

Góra sekcji ("Ludzie wchodzą", "i wychodzą") jest PRZEZROCZYSTA — tam wave-wrap animuje pod spodem.

Checklist integracji

- Wave-wrap HTML → direct child body, PRZED sekcją blok-4-5
- Wave-wrap CSS → global CSS (selektory po ID, bez parenta)
- `transition: none !important` na wave-wrap (blokada konfliktu CSS/GSAP)
- `transform: translateZ(0)` na SVG (GPU layer isolation, Safari)
- Zweryfikuj: brak `transform/will-change/filter/contain` na przodkach wave-wrap
- Bridge-wrapper/kinetic: jawny `z-index: -1` (MUSI być niższy niż wave-wrap z-index:0)
- Sekcja blok-4-5: jawny `z-index: 2` (MUSI być wyższy niż wave-wrap z-index:0)

- GSAP pin-spacer: sprawdź czy nie nadpisuje z-index na bridge — jeśli tak, użyj parametru `pinSpacer` w ScrollTrigger
- JS initWave() → zostaje w `<script>` sekcji, $id() znajdzie wave-wrap na body level
- Usuń z test shell: placeholder kinetic (div z `#2a2130`) i spacer (div height:100vh)
- Safety-bg: `.text-on-illustration-bottom` i `#blok-4-5-block-5-content` mają `background-color: #f7f6f4`
- Brak globalnego `transition: all` / `transition: opacity` na body lub wrapperach (konfliktuje z autoAlpha)

Znane edge cases (z deep research)

Safari rubber-banding (iOS)

Overscroll na iOS może powodować przeliczanie markerów ScrollTrigger na ujemnych wartościach scroll. Monitoruj zachowanie na iOS Safari — jeśli triggery strzelają podwójnie blisko krawędzi, dodaj debounce w playOpen/playClose.

Szybki scroll (fling)

Kipiel engine obsługuje to natywnie — playOpen/playClose flipują `reverse` flag, ticker kontynuuje. Nie ma tworzenia nowych instancji. Szybki scroll w dół + natychmiast w górę = animacja płynnie zmienia kierunek.

Page reload z zachowaną pozycją scroll

ScrollTrigger przy inicjalizacji oblicza stan bazując na aktualnej pozycji scroll. Jeśli user odświeży stronę będąc w połowie sekcji — ScrollTrigger wykryje że jest "za start" i odpali onEnter. Fala otworzy się bez animacji (natychmiastowe przejście do stanu open). To jest akceptowalne zachowanie.

onEnter re-fire (wielokrotne odpalanie)

onEnter strzela za KAŻDYM razem po pełnym cyklu: onEnter → (scroll dalej) → onLeave → (scroll z powrotem) → onLeaveBack → (scroll w dół) → onEnter ponownie. Domyślny `end` to `"bottom top"` — zawsze istnieje active zone.

87.